

Week 1 Applied Sheet

(Solutions)

Useful advice: The following solutions pertain to the theoretical problems given in the applied classes. You are strongly advised to attempt the problems thoroughly before looking at these solutions. Simply reading the solutions without thinking about the problems will rob you of the practice required to be able to solve complicated problems on your own. You will perform poorly on the exam if you simply attempt to memorise solutions to these problems. Thinking about a problem, even if you do not solve it, will greatly increase your understanding of the underlying concepts. Solutions are typically not provided for Python implementation questions. In some cases, pseudocode may be provided where it illustrates a particular useful concept.

Problems

Problem 1. Show, using elementary properties of the logarithm function, that the following identities are true:

- (a) $\log_2\left(\frac{k+1}{2}\right) + 1 = \log_2(k+1)$.
 (b) $a^{\log_b(n)} = n^{\log_b(a)}$ for any base $b > 1$.

Solution

- (a) We will make use of the facts that $\log_2(a) + \log_2(b) = \log_2(ab)$, and that $\log_2(2) = 1$. Using these, we find

$$\begin{aligned}\log_2\left(\frac{k+1}{2}\right) + 1 &= \log_2\left(\frac{k+1}{2}\right) + \log_2(2), \\ &= \log_2\left(\frac{k+1}{2} \times 2\right), \\ &= \log_2(k+1),\end{aligned}$$

as required.

- (b) Let's use the fact that for any positive real a , we can write $a = b^{\log_b(a)}$ since log and exponentiation are inverses. We can therefore write

$$a^{\log_b(n)} = \left(b^{\log_b(a)}\right)^{\log_b(n)}.$$

Now we dig into our exponent laws and use the fact that $(x^{p_1})^{p_2} = x^{p_1 p_2}$ to write

$$\left(b^{\log_b(a)}\right)^{\log_b(n)} = b^{\log_b(a) \log_b(n)} = \left(b^{\log_b(n)}\right)^{\log_b(a)}.$$

Finally, since $b^{\log_b(n)} = n$, we have

$$\left(b^{\log_b(n)}\right)^{\log_b(a)} = n^{\log_b(a)},$$

and hence $a^{\log_b(n)} = n^{\log_b(a)}$ as required.

Problem 2. Using mathematical induction, prove the following identity:

$$\sum_{i=1}^n i := 1 + 2 + \dots + n = \frac{n(n+1)}{2}, \quad \text{for } n \geq 1.$$

Solution

Define the following:

$$L(n) = 1 + 2 + \dots + n, \quad R(n) = \frac{n(n+1)}{2}.$$

We want to show that $L(n) = R(n)$ for all $n \geq 1$.

Base Case:

Let $n = 1$. We have

$$L(1) = 1 = \frac{1(1+1)}{2} = R(1),$$

as required.

Inductive Case:

Assume that $L(k) = R(k)$ for some $k \geq 1$, i.e. assume that

$$1 + 2 + \dots + k = \frac{k(k+1)}{2}.$$

We want to prove that $L(k+1) = R(k+1)$. Beginning with the left hand side, we wish to express the value of $L(k+1)$ in terms of $L(k)$ so that we may make use of the inductive hypothesis. We can do so by writing

$$\begin{aligned} L(k+1) &= 1 + 2 + \dots + k + (k+1), \\ &= L(k) + (k+1). \end{aligned}$$

Now for the most important step, we apply the inductive hypothesis $L(k) = R(k)$ to write

$$L(k) + (k+1) = R(k) + (k+1).$$

From here, we simply use algebra to clean up the resulting expression and obtain the desired result:

$$\begin{aligned} R(k) + (k+1) &= \frac{k(k+1)}{2} + (k+1), \\ &= \frac{k(k+1)}{2} + \frac{2(k+1)}{2}, \\ &= \frac{k(k+1) + 2(k+1)}{2}, \\ &= \frac{(k+1)(k+2)}{2}, \\ &= R(k+1), \end{aligned}$$

and therefore $L(k+1) = R(k+1)$ as required. Therefore, by induction on n , it is true that

$$\sum_{i=1}^n i := 1 + 2 + \dots + n = \frac{n(n+1)}{2}, \quad \text{for } n \geq 1.$$

Problem 3. Using mathematical induction, prove the following identity:

$$\sum_{i=0}^n r^i := 1 + r + r^2 + r^3 + \dots + r^n = \frac{r^{n+1} - 1}{r - 1}, \quad \text{for all } n \geq 0, r \neq 1.$$

Solution

Define the following:

$$L(n) = 1 + r + r^2 + r^3 + \dots + r^n, \quad R(n) = \frac{r^{n+1} - 1}{r - 1}.$$

We want to show that $L(n) = R(n)$ for all $n \geq 0$, where $r \neq 1$.

Base Case:

Let $n = 0$. We have

$$R(0) = \frac{r^{0+1} - 1}{r - 1} = \frac{r - 1}{r - 1} = 1 = L(0),$$

as required.

Inductive Case:

Assume that $L(k) = R(k)$ for some $k \geq 0$, i.e. assume that

$$1 + r + r^2 + r^3 + \dots + r^k = \frac{r^{k+1} - 1}{r - 1}.$$

We want to prove that $L(k+1) = R(k+1)$. Beginning with the left hand side, we'd like to express $L(k+1)$ in terms of $L(k)$ so that we can make use of the inductive hypothesis. We can do so by writing

$$\begin{aligned} L(k+1) &= 1 + r + r^2 + r^3 + \dots + r^k + r^{k+1}, \\ &= L(k) + r^{k+1}. \end{aligned}$$

Now we apply the inductive hypothesis that $L(k) = R(k)$ to write

$$L(k) + r^{k+1} = R(k) + r^{k+1}.$$

Now we use algebra to express the result in terms of $R(k+1)$:

$$\begin{aligned} R(k) + r^{k+1} &= \frac{r^{k+1} - 1}{r - 1} + r^{k+1}, \\ &= \frac{r^{k+1} - 1}{r - 1} + \frac{r^{k+1}(r - 1)}{r - 1}, \\ &= \frac{r^{k+1} + r^{k+1}(r - 1) - 1}{r - 1}, \\ &= \frac{r^{k+1}(1 + (r - 1)) - 1}{r - 1}, \\ &= \frac{r^{k+1}r - 1}{r - 1}, \\ &= \frac{r^{(k+1)+1} - 1}{r - 1}, \\ &= R(k+1), \end{aligned}$$

and therefore $L(k+1) = R(k+1)$ as required. Therefore, by induction on n , it is true that

$$\sum_{i=0}^n r^i := 1 + r + r^2 + r^3 + \dots + r^n = \frac{r^{n+1} - 1}{r - 1}, \quad \text{for all } n \geq 0, r \neq 1.$$

Problem 4. Using Problem 3, show that the following inequality is true for all integers $n \geq 1$

$$\sum_{i=0}^n \frac{1}{2^i} := 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^n} < 2.$$

Solution

Notice that $\sum_{i=0}^n \frac{1}{2^i}$ can be rewritten as $\sum_{i=0}^n \left(\frac{1}{2}\right)^i$, and that by setting $r = \frac{1}{2}$ we can rewrite this as $\sum_{i=0}^n r^i$, so

$$\sum_{i=0}^n r^i = \frac{r^{n+1} - 1}{r - 1} = \frac{\left(\frac{1}{2}\right)^{n+1} - 1}{\frac{1}{2} - 1} = \frac{\left(\frac{1}{2}\right)^{n+1} - 1}{-1} = 2 - \left(\frac{1}{2}\right)^n.$$

Since $\left(\frac{1}{2}\right)^n > 0$ for all $n \geq 1$, we have $2 - \left(\frac{1}{2}\right)^n < 2$, and hence it is true that $\sum_{i=0}^n \frac{1}{2^i} < 2$ as required.

Problem 5. Prove that for all $n \geq 0$,

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1.$$

Solution

Define the following:

$$L(n) = \sum_{i=0}^n 2^i = 1 + 2 + 4 + \dots + 2^n, \quad R(n) = 2^{n+1} - 1.$$

We want to show that $L(n) = R(n)$ for all $n \geq 0$.

Base Case:

Let $n = 0$. We have

$$R(0) = 2^{0+1} - 1 = 2 - 1 = 1 = L(0),$$

as required.

Inductive Case:

Assume that $L(k) = R(k)$ for some $k \geq 0$, i.e. assume that

$$1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1.$$

We want to prove that $L(k+1) = R(k+1)$. Beginning with the left hand side, we try to write $L(k+1)$ in terms of $L(k)$ so that we can make use of the inductive hypothesis.

$$\begin{aligned} L(k+1) &= 1 + 2 + 4 + \dots + 2^k + 2^{k+1}, \\ &= L(k) + 2^{k+1}. \end{aligned}$$

Now we apply the inductive hypothesis that $L(k) = R(k)$ to write

$$L(k) + 2^{k+1} = R(k) + 2^{k+1}.$$

Finally, we clean up the algebra to obtain the result.

$$\begin{aligned} R(k) + 2^{k+1} &= 2^{k+1} - 1 + 2^{k+1}, \\ &= 2 \times 2^{k+1} - 1, \\ &= 2^{(k+1)+1} - 1, \\ &= R(k+1), \end{aligned}$$

and hence $L(k+1) = R(k+1)$ as required. Therefore, by induction on n , it is true that

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1, \quad \text{for all } n \geq 0.$$

Problem 6. For the following time function, $T(n) = 3n^2 + 15\log(n) + 100n$, state whether the following statements are true or false, prove why.

- (a) $T(n) = O(n^3)$
- (b) $T(n) = \Theta(n^3)$
- (c) $T(n) = \Omega(n)$

Solution

- (a) True. Big-O just demands an upper bound on the size of a function. Here, we can select some c and n_0 , say 10 and 4 respectively, where $T(n) \leq c \cdot n^3$ for all $n \geq n_0$.
- (b) False. Though we have shown $T(n) = O(n^3)$ is true as $n \rightarrow \infty$, this is not true for $T(n) = \Omega(n^3)$. There is no choice of c and n_0 , such that $T(n) \geq c \cdot n^3$ for all $n \geq n_0$. For some large value of n , $c \cdot n^3$ will be so large such that the inequality will be false no matter the choice of c . Hence $T(n) \neq \Theta(n^3)$.
- (c) True. Big- Ω just demands some lower bound on the size of a function. Here we can select some c and n_0 , say 5 and 2 respectively, where $T(n) \geq c \cdot n$ for all $n \geq n_0$.

It is recommended you plot the functions with the appropriate c and n_0 values to confirm the inequalities hold or don't hold.

Problem 7. For each of the following expressions, write the Big- Θ function corresponding to it:

- (a) $3n^3 + 100n^2 + n$.
- (b) $n^3 \log(n) + 0.5n^4 + 100n^2$.
- (c) $5\sqrt{n} + 10\log(n)$.
- (d) $\log(n) + \log(\log(n)) + \log(\log(\log(n)))$.
- (e) $2n \log(n) + 8n \log(n^2)$.
- (f) $3n \log(n) + 5n(\log(n))^2$.
- (g) $\log(n) + 2\log^2(n)$.
- (h) $3\log(n) + (\log(\log(n)))^2$.

Solution

- (a) As n gets large, n^3 dominates n^2 and n , so this is $\Theta(n^3)$.
- (b) First, n^4 dominates n^2 . Second, we know that n dominates $\log(n)$, so consequently n^4 dominates $n^3 \log(n)$, so this is $\Theta(n^4)$.
- (c) $\sqrt{n}$ is equivalent to $n^{\frac{1}{2}}$. We know that for any value of $k > 0$, n^k dominates $\log(n)$ (if you've taken MTH1030, try to prove this formally – that is $\lim_{n \rightarrow \infty} \frac{\log(n)}{n^k} = 0$), so this is $\Theta(\sqrt{n})$.
- (d) Since $\log(n)$ is dominated by n , having many nested logs is smaller than a single log, therefore this is $\Theta(\log(n))$.
- (e) Since $\log(n^2) = 2\log(n)$, both terms are the same magnitude $\Theta(n \log(n))$, hence the answer is $\Theta(n \log(n))$.

- (f) Since $\log(n)$ is not $O(1)$, $(\log(n))^2$ dominates $\log(n)$, hence this is $\Theta(n(\log(n))^2)$.
- (g) The notation $\log^2(n)$ is short for $(\log(n))^2$, so by the same reasoning as above, this is $\Theta(\log^2(n))$. Note that this notation does not mean $\log(\log(n))$.
- (h) Intuitively, we understand that $\log(n)$ is the inverse exponential, hence it grows slower than the square function grows fast. This should lead you to believe that $\log(n)$ dominates $(\log(\log(n)))^2$. To see this more concretely, substitute $n = 2^{2^k}$ and we obtain $3 \cdot 2^k + k^2$, and it is clear that the first term dominates, hence this is $\Theta(\log(n))$.

For further learning you should formally utilise the definition of Big- Θ choosing a c and n_0 . Intuitively though it is just about selecting the most dominant (fastest growing) term.

Problem 8. Here we are going to analyse the time complexity of `CountTotalBits` and `CountBits`. The algorithms are defined as such.

```

1: function COUNTTOTALBITS(arr[1...n])
2:   totalBits = 0
3:   for i = 1 to n do
4:     totalBits = totalBits + COUNTBITS(arr[i])
5:   return totalBits

```

and

```

1: function COUNTBITS(x)
2:   bits = 1
3:   while x > 1 do
4:     bits = bits + 1
5:     x = ⌊x/2⌋
6:   returns bits

```

State the Big- Θ complexity for `CountTotalBits` for each of the following scenarios:

- (a) The length `arr` is n and $0 \leq \text{arr}[i] \leq 2^m - 1$
- (b) The length `arr` is n where $1 \leq n \leq 1000000$ and $0 \leq \text{arr}[i] \leq 2^m - 1$
- (c) The length `arr` is n and $0 \leq \text{arr}[i] \leq 2^{32} - 1$

Solution

- (a) $\Theta(mn)$. The array has n total elements, meaning the for loop in `CountTotalBits` will run n times and therefore `CountBits` will be called n times. The time of `CountBits` is proportional to how many bits an input number has. The input has a specified range of $0, \dots, 2^m - 1$, which means the max bits of any input is m , thus `CountBits` costs $\Theta(m)$. If you don't understand why `CountBits` cost the number of bits of some integer, you can imagine that if you have some number that is some power of 2, dividing by 2 each time reduces the power by 1. For the number 2^m to eventually come down to 1, you would need to repeat the division m times.
- (b) $\Theta(m)$. As per (a) `CountBits` cost $\Theta(m)$. The loop for `CountTotalBits` is executed n times which is bounded by 1000000. Therefore exists a c and lets say 1000001, such that $n \leq c \cdot 1$ is true. Hence there is $O(1)$ total loop executions and therefore constant amount of calls to `CountBits`. A function called a constant amount of times is same Big-O cost of calling the function once.
- (c) $\Theta(n)$. As per (a) n calls are made to `CountBits`. The cost of each `CountBits` call is now only $O(1)$ as there is a constant amount of bits a number can have if we limit the range to $0, \dots, 2^{32} - 1$. That

is there is at most 32 bits for any input.

Problem 9. The harmonic numbers are defined as the partial sums of the harmonic sequence:

$$H(n) := \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}.$$

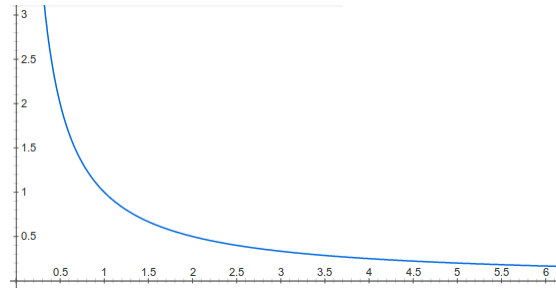
We want to determine how this quantity behaves asymptotically, as it frequently arises in algorithm analysis.

- (a) Prove that $H(n) > \log_e(n)$
- (b) Prove that $H(n) \leq \log_e(n) + 1$
- (c) Hence deduce that $H(n) = \Theta(\log(n))$ ¹

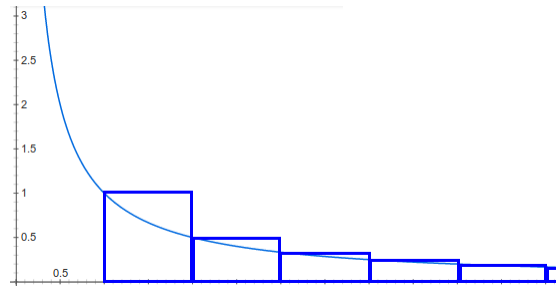
[Hint: Consider the area under the function $f(x) = \frac{1}{x}$ using calculus and compare this with $H(n)$ geometrically.]

Solution

Consider the function $f(x) = \frac{1}{x}$ for $x \geq 1$. A plot of f is depicted below.



We notice that the value of the sum $H(n) = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$ is equal to the area of the rectangles depicted on the following plot.



Since the rectangles are above the curve, their total area is bigger than the area underneath the curve (from $x = 1$ onwards). The area underneath the curve is given by

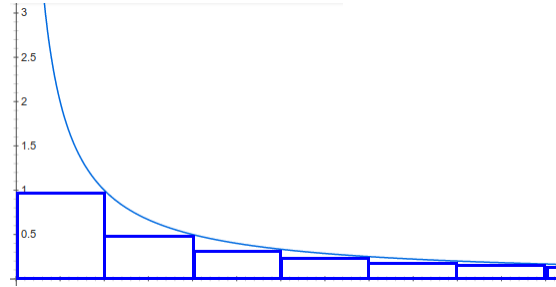
$$\int_1^{n+1} \frac{1}{x} dx = \log_e(n+1),$$

and therefore we have

$$H(n) > \log_e(n+1) > \log_e(n),$$

and thus $H(n) > \log_e(n)$. Now, consider the same rectangles but all shifted 1 unit to the left.

¹Remember that the notation $\Theta(f(n))$ means that $H(n)$ is both upper bounded by $c_1 f(n)$ for some c_1 (it is $O(f(n))$) and lower bounded by $c_2 f(n)$ for some c_2 (it is $\Omega(f(n))$).



The leftmost rectangle has area 1, and the others all are all below the curve. The area underneath the curve from 1 onwards is given by

$$\int_1^n \frac{1}{x} = \log_e(n),$$

and therefore we have

$$H(n) \leq 1 + \log_e(n),$$

and thus we have $\log_e(n) < H(n) \leq \log_e(n) + 1$. Together, we deduce that $H(n) = \Theta(\log(n))$.

Problem 10. The Fibonacci sequence is defined by the recurrence relation $F(n) = F(n-1) + F(n-2)$, with $F(1) = F(2) = 1$.

- Write a Python function that, given a non negative integer n , computes the n^{th} Fibonacci number iteratively (using a loop). Analyse the time and space complexity of your implementation.
- Write a Python function that, given a non negative integer n , computes the n^{th} Fibonacci number recursively. Analyse the time and space complexity of your implementation.

Think carefully about the assumptions that you make when analysing the time and space complexity of your algorithms.